

Bringing MNIST Handwriting Classification Accuracy to 99%+

Explore a progressive journey of architectural and training innovations designed to systematically enhance model capabilities and push performance boundaries.



Baseline: Multi-Layer Perceptron



Fully-Connected Neural Network

The Traditional Approach

How It Works

- 1 Flatten Image**
Convert 28×28 pixels into 784-dimensional vector
- 2 Fully-Connected Layers**
Every neuron connects to all neurons in next layer
- 3 Output Classification**
Produce probabilities for 10 digit classes

⚠ Critical Limitation

Loses Spatial Relationships

- ✗ No notion of pixel adjacency
- ✗ Ignores local patterns like edges
- ✗ Cannot exploit 2D structure

📈 Performance

96-97%

Reasonable baseline, but hits fundamental limits

Experiment 1: Convolutional Neural Networks

Preserving Spatial Structure

Learn features directly from 2D image data

Two Key Concepts

Convolution

Sliding filters detect patterns (edges, curves, textures)

Pooling

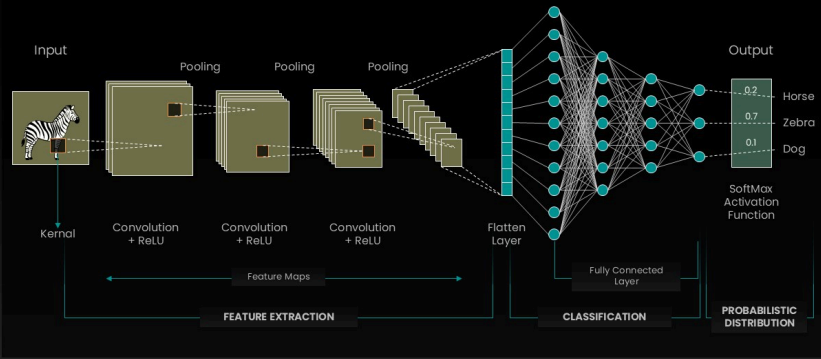
Downsampling reduces dimensions, translation invariance

 Performance

~98%

Convolutional Neural Networks – CNN

PowerPoint Template



Hierarchical Feature Learning

- **Early Layers**
Simple features: edges, corners
- **Middle Layers**
Combination features: shapes, patterns
- **Deep Layers**
Complex features: digit components

Experiment 2: Regularization Techniques

Batch Normalization

Normalize Layer Activations

- ✓ Zero mean, unit variance per batch
- ✓ Reduces internal covariate shift
- ✓ Stabilizes training process
- ✓ Enables higher learning rates

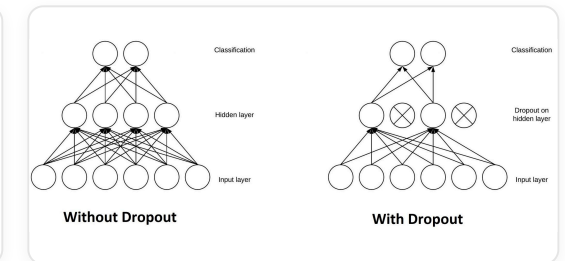
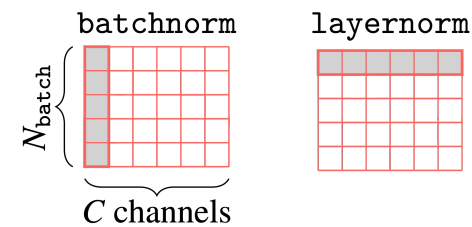
 Performance

~98.5%

Dropout

Random Neuron Deactivation

- ✓ Drop neurons with probability p
- ✓ Forces robust feature learning
- ✓ Prevents neuron co-adaptation
- ✓ Acts as ensemble of subnetworks



Experiment 3: Data Augmentation

Synthetic Data Generation

Generate variations from existing training images

Why It Works

- ✓ Increases effective training data size
- ✓ Prevents memorization/overfitting
- ✓ Simulates real-world writing variations

 Performance

~99%

Applied Transformations



Random Rotation

$\pm 10^\circ$ rotation simulates different writing angles



Random Translation

$\pm 10\%$ shift simulates digit positioning



Each epoch sees different transformed versions



Rotation

$\pm 10^\circ$



Translation

$\pm 10\%$



Every Epoch

New Variants

Experiment 4: Learning Rate Scheduling

Adaptive Learning Rate

Decay learning rate during training for better convergence

Two-Phase Strategy

High LR Initial Phase

Explores solution space broadly, moves quickly

Lower LR Later Phase

Fine-tunes solution, converges precisely

 Performance **~99.2%**

Adaptive Learning Dynamics

 Epochs 1-2	0.01
 Epochs 3-4	0.007
 Epochs 5-6	0.0049
 Epochs 7+	0.0034

Progressive Decay Strategy

Learning rate multiplies by 0.7 every 2 epochs for optimal convergence


Start Fast


Gradual Decay


End Precise

Experiment 5: ResNet-style Architecture

Residual Connections

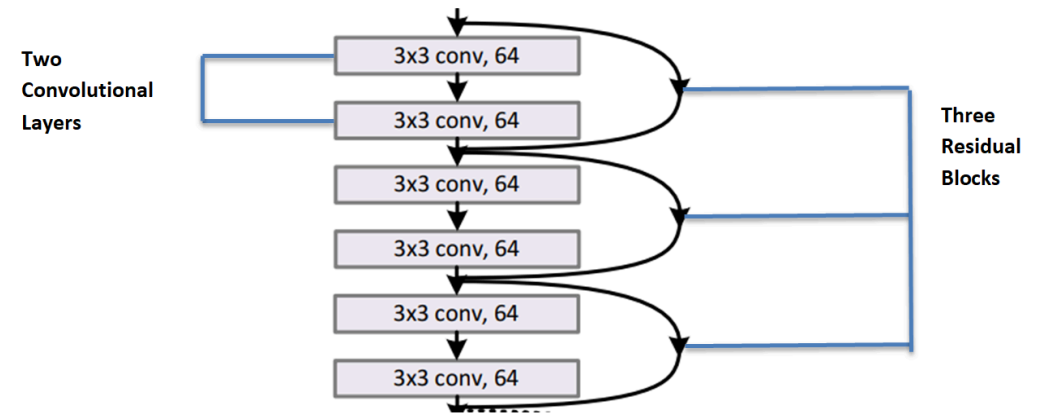
Skip connections enable deeper networks

The Problem

- ⚠ Vanishing gradients in deep networks
- ⚠ Early layers stop learning
- ⚠ Hard to train beyond ~20 layers

📈 Performance

~99.3%+



The Solution

➡ Skip Connections

Identity mapping allows direct gradient flow

Σ Residual Learning

Learn changes: $H(x) = F(x) + x$

🚀 Enables training 152+ layer networks

Experiment 6: Deeper CNN Architecture

Increased Depth & Capacity

Stacking more convolutional layers for complex pattern learning

Key Improvements

- ✓ More layers = more representational capacity
- ✓ Learn abstract, complex hierarchical features
- ✓ Capture patterns at multiple scales

 Performance **~99.5-99.6%**

Architecture Details

Block 1	Block 2	Block 3	Block 4
32	64	128	256

Expanding channels captures features at multiple scales

Global Average Pooling

- ✓ Reduces parameters dramatically
- ✓ Prevents overfitting
- ✓ Better than flattening

 Trade-off: capacity vs. overfitting risk